

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..20

CUSTOMIZING THE UI

 Copy as Markdown

Customize the Firebase authentication UI

When using the Firebase identity provider, you build your authentication UI on top of `firebase_auth`, optionally with `firebase_ui_auth` for the pre-built sign-in screens. The `FirebaseAuthController` handles syncing the authenticated Firebase user with your Serverpod backend. This page breaks down the gate widget shown in the [setup guide](#) and then covers building a fully custom UI with `firebase_auth` directly.

Anatomy of the gate widget

The [setup guide](#) shows a `SignInScreen` widget that wraps your app and only renders the `child` once the user is authenticated. Three pieces drive that flow:

FirebaseAuthController

The controller manages the sync between Firebase authentication state and your Serverpod session. The `client` argument is the global Serverpod `Client` instance created in `lib/client.dart` (see [Initialize Firebase and Serverpod](#)).

```
controller = FirebaseAuthController(  
  client: client,  
  onAuthenticated: () => context.showSnackBar(  
    message: 'User authenticated.',  
    backgroundColor: Colors.green,  
  ),  
  onError: (error) => context.showSnackBar(  
    message: 'Authentication failed: $error',  
    backgroundColor: Colors.red,  
  ),  
);
```

 **WARNING**


Ask AI

Do not navigate to a home screen inside `onAuthenticated`. This callback fires every time a sync succeeds, including on app restart. Instead, gate the UI on `controller.isAuthenticated` (as the setup example does), or listen to `client.auth.authInfoListenable`.

Action handlers

The `firebase_ui_auth` package emits state changes through `AuthStateChangeAction`. Two handlers cover the cases that need a Serverpod sync:

- `SignedIn`: a returning user signed in.
- `UserCreated`: a brand-new account was created.

Both call `controller.login(user)` so the Firebase user is registered with Serverpod:

```
Future<void> _handleAuthStateChange() async {  
  final user = firebase_auth.FirebaseAuth.instance.currentUser;  
  if (user != null) await controller.login(user);  
}
```

`controller.isAuthenticated`

The `build` method checks `controller.isAuthenticated` to decide whether to show the sign-in UI or the wrapped `child`:

```
if (controller.isAuthenticated) return widget.child;
```

Add more providers to the `firebase_ui.SignInScreen` as needed (see the [firebase_ui_auth documentation](#)):

```
providers: [  
  firebase_ui.EmailAuthProvider(),  
  // firebase_ui.PhoneAuthProvider(),  
  // firebase_ui.GoogleProvider(clientId: '...'),  
],
```

Controller reference

The login method

After a user signs in through Firebase (using any method), pass the `firebase_auth.User` to the controller to sync with Serverpod:

```
final firebaseUser = firebase_auth.FirebaseAuth.instance.currentUser;
```

```
if (firebaseUser != null) {  
  await controller.login(firebaseUser);  
}
```

State management

The controller is a `ChangeNotifier`, so it notifies your widget whenever its state changes. Register a listener inside `initState` and call `setState` to trigger a rebuild:

```
@override  
void initState() {  
  super.initState();  
  controller = FirebaseAuthController(client: client);  
  
  controller.addListener(() {  
    if (mounted) setState(() {});  
  });  
}
```

The controller exposes a few properties for your `build` method:

- `controller.isLoading`: Whether the controller is processing a request.
- `controller.isAuthenticated`: Whether the user is authenticated.
- `controller.errorMessage`: The error message string, if any.
- `controller.error`: The raw error object, for advanced error handling.

Use them in your `build` method to render the right UI for the current state:

```
@override  
Widget build(BuildContext context) {  
  if (controller.isLoading) {  
    return const Center(child: CircularProgressIndicator());  
  }  
  
  if (controller.errorMessage != null) {  
    return Center(child: Text('Error: ${controller.errorMessage}'));  
  }  
  
  if (controller.isAuthenticated) {  
    return widget.child;  
  }  
  
  // Otherwise, show the sign-in UI.  
  return const SignInForm();  
}
```

If you need finer-grained control, switch on `controller.state` directly:

```
@override
Widget build(BuildContext context) {
  return switch (controller.state) {
    FirebaseAuthState.idle => const SignInForm(),
    FirebaseAuthState.loading => const Center(child: CircularProgressIndicator()),
    FirebaseAuthState.error => Center(child: Text('Error: ${controller.errorMessage}')),
    FirebaseAuthState.authenticated => widget.child,
  };
}
```

Using firebase_auth directly

For full control over the authentication UI, use the `firebase_auth` package directly. Build your own sign-in UI, call `FirebaseAuth` methods, then pass the result to `controller.login()`:

```
import 'package:firebase_auth/firebase_auth.dart' as firebase_auth;

final credential = await firebase_auth.FirebaseAuth.instance
  .signInWithEmailAndPassword(
    email: emailController.text,
    password: passwordController.text,
  );

final user = credential.user;
if (user != null) {
  await controller.login(user);
}
```

Refer to the [firebase_auth documentation](#) for all available authentication methods.

Listening to Firebase auth state changes

For apps that need to react to Firebase auth state changes automatically (e.g., the user signs in on another device, or the session is restored after an app restart), subscribe to

`FirebaseAuth.instance.authStateChanges()`. The stream emits the current `User` whenever Firebase's auth state changes; pass that user to `controller.login()` to sync with Serverpod.

```
firebase_auth.FirebaseAuth.instance.authStateChanges().listen((user) async {
  if (user == null) return;

  // Guard against re-syncing an already authenticated user.
  if (!controller.isAuthenticated) {
    await controller.login(user);
  }
});
```



WARNING

Always check `controller.isAuthenticated` before calling `login()` inside the listener. The stream fires on every app start with the cached Firebase user, and `login()` would otherwise re-sync on every launch.